

Documentación Técnica: Algoritmo de Cola de Impresiones

Este proyecto universitario implementa una Cola (Queue) con el objetivo de gestionar un flujo de trabajo (workflow) bajo la metodología FIFO (First In, First Out).

1. Definición de la Estructura de Datos

La clase Queue actúa como el contenedor principal de los datos, garantizando que el orden de llegada de los elementos se respete estrictamente durante todo el ciclo de vida de la aplicación.

- **Fundamento Lógico:** Se basa en una estructura lineal donde los elementos se insertan por un extremo (*rear*) y se eliminan por el extremo opuesto (*front*).
- **Operaciones Implementadas:**
 - enqueue(item): Inserta un nuevo elemento al final de la estructura.
 - dequeue(): Elimina y retorna el elemento situado en el frente.
 - front(): Consulta el elemento frontal sin modificar la estructura.
 - is_empty(): Retorna un valor booleano indicando el estado de la cola.
 - size(): Retorna el conteo actual de elementos almacenados.

2. Lógica del Algoritmo de Gestión

El sistema para el control de trabajos de impresión se articula mediante tres procesos fundamentales:

1. **Inserción:** El sistema captura los datos del usuario y los encapsula en un objeto de clase TrabajoImpresion, el cual es enviado a la cola. Cada trabajo recibe un ID consecutivo para su trazabilidad.
2. **Visualización (Algoritmo de Muestreo):** Debido a que la naturaleza de una cola es retirar elementos para acceder a ellos, se implementó un proceso de vaciado temporal:
 - Se desencolan todos los elementos para obtener sus datos.
 - Se insertan en la interfaz gráfica (QTableWidget).
 - Se reencolan inmediatamente para restaurar la cola original sin pérdida de datos.
3. **Procesamiento:** Se ejecuta el método dequeue() para retirar el elemento que ha permanecido más tiempo en espera, simulando la atención del servicio de impresión.

3. Consideraciones Técnicas

- **Encapsulamiento:** Se utilizó un GestorImpresion como capa intermedia para desacoplar la lógica de la interfaz gráfica (UI) de la lógica de la estructura de datos.
- **Interfaz:** La UI fue desarrollada con PyQt5, permitiendo una comunicación síncrona entre las acciones del usuario y el estado de la estructura de datos mediante el método actualizar_interfaz(), que se ejecuta tras cada evento de inserción o eliminación.